

RV32IM + PCPI

A Mano-Style Register-Transfer Specification

Base ISA · M-Extension · Custom PCPI Coprocessor Interface (RELU)

Scope

- Full RV32I base instruction set
 - RV32M extension: MUL / MULH / MULHSU / MULHU / DIV / DIVU / REM / REMU
 - Custom PCPI coprocessor instruction (RELU) on the custom-0 opcode
-

Overview

This document specifies a single-cycle-per-microoperation (Mano-style) RTL design implementing the RV32I base integer ISA, the RV32M multiply/divide extension[\[1\]](#), and a custom coprocessor interface (PCPI) demonstrating one user-defined instruction, RELU.

Each machine cycle is expressed as a sequence of timing states (T0, T1, T2, ...), grouped into three phases: **Fetch**, **Decode**, and **Execute**, consistent with classical Mano-style datapath design. The Execute phase branches into decode classes D0–D10, one per instruction format, selected by the top-level opcode decoder.

Datapath Registers

Register	Width	Purpose
PC	32 bits	Program counter
PC_SAVE	32 bits	PC snapshot latched at fetch, used for branch/JAL/AUIPC target computation
MAR	32 bits	Memory address register
MDR	32 bits	Memory data register

IR	32 bits	Instruction register
A, B	32 bits each	Operand latches, loaded from RF[RS1], RF[RS2] at decode
ALU_OUT	32 bits	ALU result latch
MUL_OUT	32 bits	Multiply/divide result latch (M-extension); holds one 32-bit half of the 64-bit product for MULH/MULHSU/MULHU, or the full 32-bit result for MUL/DIV/DIVU/REM/REMU
RF	32 × 32 bits	General-purpose register file (x0–x31)
COUNTER	4 bits	Sequencer state register; encodes T0–T15; COUNTER <- T0 returns control to Fetch
PCPI_VALID	1 bit	Asserted by the core to signal a pending coprocessor instruction
PCPI_A, PCPI_B	32 bits each	Operand copies dispatched to the coprocessor
PCPI_IR	32 bits	Instruction word copy dispatched to the coprocessor
PCPI_R	32 bits	Result returned from the coprocessor
PCPI_WAIT	1 bit	Asserted by the coprocessor while its result is not yet valid
PCPI_READY	1 bit	Pulsed by the coprocessor when PCPI_R is valid

1. Fetch

Common to every instruction. Loads the instruction addressed by PC into IR and advances PC, while preserving the original PC in PC_SAVE for later use by branch, jump, and AUIPC targets.

```

T0: MAR ← PC, PC_SAVE ← PC
T1: MDR ← MEM[MAR], PC ← PC+4
T2: IR ← MDR

```

2. Decode

Register operands are read speculatively for every instruction; classes that don't need A/B (e.g. LUI) simply don't consume them.

```
T3: A ← RF[RS1], B ← RF[RS2]
```

2.1 Top-Level Opcode Decoder

The sequencer branches into D0–D10 purely on the RV32 opcode field (IR[6:0]); sub-fields (funct3, funct7) refine behavior within a class.

Opcode	Hex	Class	Decode	Sub-decode
0110011	0x33	R-type ALU	D0	funct7 == 0x00/0x20 → D0
0110011	0x33	M-extension	D1	funct7 == 0x01 → D1 (same opcode as D0)
0010011	0x13	I-type ALU	D2	funct3 selects op; funct7[5] splits SRLI/SRAI
0000011	0x03	LOAD	D3	funct3 selects LB/LH/LW/LBU/LHU
0100011	0x23	STORE	D4	funct3 selects SB/SH/SW
1100011	0x63	BRANCH	D5	funct3 selects BEQ/BNE/BLT/BGE/BLTU/BGEU
1101111	0x6F	JAL	D6	—
1100111	0x67	JALR	D7	funct3 == 0x00
0110111	0x37	LUI	D8	—
0010111	0x17	AUIPC	D9	—

0001011	0x0B	custom-0 (PCPI / RELU)	D10	reserved for future custom sub-ops
---------	------	---------------------------	-----	------------------------------------

Design note, opcode-only decode and the R-type / M-extension split:

R-type ALU (D0) and the M-extension (D1) share opcode 0x33. The sequencer first branches on opcode; funct7[0] then selects between D0 and D1. This keeps the top-level decoder purely opcode-driven while still distinguishing the two classes before the ALU/multiplier datapath fork.

```

case (opcode)
    7'b0110011: (funct7[0] == 1'b1) ? D1 : D0    // R-type ALU / M-ext
    7'b0010011: D2                               // I-type ALU
    7'b0000011: D3                               // LOAD
    7'b0100011: D4                               // STORE
    7'b1100011: D5                               // BRANCH
    7'b1101111: D6                               // JAL
    7'b1100111: D7                               // JALR
    7'b0110111: D8                               // LUI
    7'b0010111: D9                               // AUIPC
    7'b0001011: D10                             // custom-0 (RELU)
    default:    ILLEGAL_INSTR
endcase

```

3. Execute

Each decode class begins at T4 (after common Fetch/Decode cycles T0–T3) and ends by reloading COUNTER <- T0, returning control to Fetch.

D0 - R Type ALU (opcode 0x33, funct7 ≠ 0x01)

T4:

```

funct3=0:
    funct7[5]': ALU_OUT <- A + B
    funct7[5]: ALU_OUT <- A - B
funct3=1: ALU_OUT <- A << B[4:0]
funct3=2: ALU_OUT <- (A < B), signed
funct3=3: ALU_OUT <- (A < B), unsigned
funct3=4: ALU_OUT <- A xor B
funct3=5:
    funct7[5]': ALU_OUT <- A >> B[4:0]

```

```

        funct7[5]: ALU_OUT <- A >>> B[4:0]
    funct3=6: ALU_OUT <- A or B
    funct3=7: ALU_OUT <- A and B
T5: RF[RD] <- ALU_OUT
T6: COUNTER <- T0

```

D1 - M Extension: Multiply / Divide (opcode 0x33, funct7 = 0x01)

```

T4:
    funct3=0: MUL_OUT <- ($A * $B)[31:0]      MUL
    funct3=1: MUL_OUT <- ($A * $B)[63:32]     MULH (signed x signed)
    funct3=2: MUL_OUT <- ($A * B)[63:32]      MULHSU (signed x unsigned)
    funct3=3: MUL_OUT <- (A * B)[63:32]       MULHU
    funct3=4: MUL_OUT <- $A / $B              DIV
    funct3=5: MUL_OUT <- A / B                DIVU
    funct3=6: MUL_OUT <- $A % $B              REM
    funct3=7: MUL_OUT <- A % B                REMU
T5: RF[RD] <- MUL_OUT
T6: COUNTER <- T0

```

Design note — divide-by-zero and overflow are architecturally defined:

- DIV by zero → result = -1 (all ones). DIVU by zero → result = 0xFFFFFFFF.
- REM/REMU by zero → result = the dividend, unchanged.
- INT_MIN / -1 overflow → DIV returns INT_MIN; REM returns 0.
- These cases must be special-cased in the divider datapath - RISC-V defines them precisely rather than leaving them undefined^[1], unlike C's UB on the same operations.

D2 - I Type ALU (opcode 0x13)

```

T4:
    funct3=0: ALU_OUT <- A + IMM(I)
    funct3=1: ALU_OUT <- A << IMM(I)[4:0]
    funct3=2: ALU_OUT <- (A < IMM(I)), signed
    funct3=3: ALU_OUT <- (A < IMM(I)), unsigned
    funct3=4: ALU_OUT <- A xor IMM(I)
    funct3=5:
        funct7[5]': ALU_OUT <- A >> IMM(I)[4:0]
        funct7[5]: ALU_OUT <- A >>> IMM(I)[4:0]
    funct3=6: ALU_OUT <- A or IMM(I)
    funct3=7: ALU_OUT <- A and IMM(I)
T5: RF[RD] <- ALU_OUT
T6: COUNTER <- T0

```

D3 - LOAD (opcode 0x03)

```
T4: ALU_OUT <- A + IMM(I)
T5: MAR <- ALU_OUT
T6: MDR <- MEM[MAR]
T7: RF[RD] <- MDR    (width/sign-extended per funct3: LB/LH/LW/LBU/LHU)
T8: COUNTER <- T0
```

D4 - STORE (opcode 0x23)

```
T4: ALU_OUT <- A + IMM(I)
T5: MAR <- ALU_OUT, MDR <- RF[RS2]
T6: MEM[MAR] <- MDR    (via MEM_STRB, width per funct3: SB/SH/SW)
T7: COUNTER <- T0
```

D5 - BRANCH (opcode 0x63)

```
T4:
    funct3 in {0,1} (BEQ/BNE): ALU_OUT <- A - B
    funct3 in {4,5} (BLT/BGE): ALU_OUT <- (A < B), signed
    funct3 in {6,7} (BLTU/BGEU): ALU_OUT <- (A < B), unsigned
T5:
    funct3=0 (BEQ), Z true: PC <- PC_SAVE + IMM(B)
    funct3=1 (BNE), Z' true: PC <- PC_SAVE + IMM(B)
    funct3=4 (BLT), Z' true: PC <- PC_SAVE + IMM(B)
    funct3=5 (BGE), Z true: PC <- PC_SAVE + IMM(B)
    funct3=6 (BLTU), Z' true: PC <- PC_SAVE + IMM(B)
    funct3=7 (BGEU), Z true: PC <- PC_SAVE + IMM(B)
T6: COUNTER <- T0
```

D6 - JAL (opcode 0x6F)

```
T4: RF[RD] <- PC
T5: PC <- PC_SAVE + IMM(J)
T6: COUNTER <- T0
```

D7 - JALR (opcode 0x67)

```
T4: RF[RD] <- PC, ALU_OUT <- A + IMM(I)
T5: PC <- ALU_OUT    (LSB forced to 0)
T6: COUNTER <- T0
```

D8 - LUI (opcode 0x37)

```
T4: RF[RD] <- IMM(U)
T5: COUNTER <- T0
```

D9 - AUIPC (opcode 0x17)

```
T4: ALU_OUT <- PC_SAVE + IMM(U)
T5: RF[RD] <- ALU_OUT
T6: COUNTER <- T0
```

D10 - PCPI: Custom Coprocessor Instruction, RELU (opcode 0x0B, custom-0)

Note on PCPI itself: PCPI (Pico Co-Processor Interface) is not part of the official RISC-V specification. The RISC-V spec only reserves the custom-0/1/2/3 opcode encodings and leaves their behavior fully undefined for the implementer. PCPI is a convention originating in PicoRV32 (Clifford Wolf's open-source RV32 core)[\[2\]](#) for how a core hands off a custom instruction to an external coprocessor. This design adopts that convention as its coprocessor handshake.

D10 demonstrates a minimal PCPI-style handoff: the core dispatches operands and the instruction word to an external coprocessor, waits for a result, then retires it into the register file.

PCPI signal set (matches PicoRV32):

Signal	Direction	Meaning
PCPI_VALID	core → coprocessor	Operands are ready; this is a PCPI instruction
PCPI_IR	core → coprocessor	Full instruction word
PCPI_A, PCPI_B	core → coprocessor	Operands
PCPI_WAIT	coprocessor → core	Not done yet — keep stalling
PCPI_READY	coprocessor → core	Result is valid this cycle
PCPI_R	coprocessor → core	Result value

```
T4: PCPI_VALID <- 1, PCPI_A <- A, PCPI_B <- B, PCPI_IR <- IR
T5: PCPI_R <- RELU(PCPI_A)                [inside coprocessor]
    if PCPI_READY' : hold at T5           (coprocessor not done -
stall)
    if PCPI_READY  : advance to T6
T6: RF[RD] <- PCPI_R, PCPI_VALID <- 0
T7: COUNTER <- T0
```

Design note - this assumes single cycle RELU, and that's a stated simplification, not an oversight:

Since **RELU** ($\max(0, x)$) is purely combinational, T5 in practice resolves in one cycle and PCPI_READY is effectively tied high. The PCPI_WAIT/PCPI_READY handshake is included anyway because it's the general mechanism the interface needs to support: any coprocessor with variable or multi-cycle latency (e.g. a multi-cycle MAC or a pipelined multiplier) stalls the core at T5 until PCPI_READY pulses, rather than the core guessing a fixed number of cycles and advancing regardless. This is also how PicoRV32 itself behaves[\[2\]](#) - the core pipeline genuinely stalls (does not fetch the next instruction) while a PCPI op is outstanding.

Design note - unclaimed PCPI instructions need a timeout: if PCPI_VALID is asserted but no attached coprocessor ever responds with PCPI_READY (e.g. the instruction's funct3/funct7 doesn't match any implemented coprocessor op), the FSM would stall at T5 indefinitely. PicoRV32 handles this with a cycle-count timeout that fires an illegal-instruction trap if no coprocessor claims the instruction. This design doesn't yet implement that timeout, it's listed as a future extension below rather than silently assumed away.

3.1 Custom Opcode Reservation

The RISC-V specification reserves four opcode blocks explicitly for vendor and user-defined instructions[\[1\]](#), guaranteed never to collide with standard extensions. RELU is assigned to custom-0; the remaining three are available for future coprocessor instructions without any change to the top-level decoder structure.

Block	Encoding
custom-0	0001011 (0x0B)
custom-1	0101011 (0x2B)
custom-2	1011011 (0x5B)
custom-3	1101011 (0x7B)

Design note - PCPI as a general extension mechanism, not a one-off:

RELU exercises the full PCPI handshake - dispatch, compute, retire - in the minimum number of states. That's what makes it a good first demo: it proves the interface works before adding the complexity of a multi-cycle coprocessor operation.

A follow-on instruction that reads from memory, such as a multiply-accumulate over a vector, would need two things RELU doesn't: an explicit memory-latency state between issuing a read ($\text{MAR} \leftarrow$) and consuming it ($\text{op} \leftarrow \text{MDR}$), matching the two-cycle pattern already used in Fetch (T0/T1); and address/length counters that update each loop pass, since looping back to a fixed state alone will re-read the same address indefinitely.

References

- [1] RISC-V International. "The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA."
<https://riscv.org/technical/specifications/>
- [2] Wolf, Clifford. PicoRV32 — A Size-Optimized RISC-V CPU.
<https://github.com/YosysHQ/picorv32> (see the "Pico Co-Processor Interface (PCPI)" section of the README)